NannyMom

Technical Architecture — v3.0

React Native · Node.js / TypeScript · AWS API Gateway

Paymob Payments · AWS Cloud · Turborepo Monorepo

March 2026 | Confidential

1. Turborepo Monorepo — Approach

The entire NannyMom project lives in a single Turborepo monorepo — one repository containing mobile, backend, infrastructure, and shared packages. This is the single most important structural decision for agentic development.

> Agentic Development Principle: Claude Code operates with full context in a single session across the entire codebase. A change to a shared type immediately surfaces type errors in mobile, backend, and infrastructure code simultaneously — catching breaking changes before they reach CI, with no context-switching between repositories.

Area	Location	Contents
Mobile app	apps/mobile/	React Native — Expo bare workflow, all screens, offline sync, WebRTC client
Backend	apps/backend/	Node.js modular monolith — all 12 business modules + WebSocket hub in one deployable (auth, booking, nanny, care-log, chat, video, community, marketplace, wallet, notification, support, admin)
Infrastructure	infra/	AWS CDK in TypeScript — all AWS resources defined as code, same language as the app
Shared packages	packages/	TypeScript types, DB schema, Paymob SDK, API client, UI components — consumed by both apps
CI/CD	.github/workflows/	GitHub Actions — Turborepo-aware, only builds and tests what changed

Monorepo Benefit	How It Applies to NannyMom
Single language	TypeScript across mobile, backend, infrastructure, and shared packages — one mental model, one toolchain
Shared types	BookingStatus, PaymobOrder, CareLogEntry defined once in packages/types — used identically in mobile UI and backend API
Atomic commits	A feature that touches mobile screen + backend endpoint + DB schema ships as one PR — no cross-repo coordination

Monorepo Benefit	How It Applies to NannyMom
Turborepo caching	Only affected packages are rebuilt and tested on each PR — fast CI even as the codebase grows
Claude Code efficiency	Full codebase in one context window — AI agent can implement a complete feature end-to-end in a single session
CDK + app together	Infrastructure changes and the code that runs on that infrastructure live side by side — no drift between what's deployed and what's defined

1.1 Agentic Development with Claude Code

Because the entire codebase lives in a single TypeScript repo, Claude Code can implement complete features end-to-end in one session — from database schema to backend API to mobile screen — without switching repositories, languages, or tools. Three representative examples:

Example	What happens in one Claude Code session
Add a new feature (e.g. nanny availability calendar)	Claude adds the shared type to packages/types, adds the DB table to packages/db and generates the migration, implements the backend endpoint in apps/backend, generates the typed API client in packages/api-client, and builds the mobile screen in apps/mobile — TypeScript compile errors guide every step and nothing is left inconsistent
DB schema refactor (e.g. split a table, rename a column)	Claude edits the schema in packages/db, runs the migration generator, then follows every TypeScript error across apps/backend and apps/mobile to update all affected queries and UI bindings in one pass — no manual grep, no missed callsites
Provision a new AWS resource (e.g. add an SQS queue for async notifications)	Claude adds the CDK construct in infra/lib/stacks/, wires the queue ARN into the backend via Secrets Manager or environment config, updates the backend module to publish to the queue, and opens a PR — infrastructure and application code change together in one atomic commit

1.1a Context Management — Markdown Files

Claude Code's context window is the source of truth for an agentic session. Structured Markdown files at the repo root provide Claude with persistent, version-controlled context that survives across sessions and across team members. Every file is committed to the monorepo and updated as the project evolves.

File	Purpose & What Claude reads from it
CLAUDE.md	Root-level system prompt for the entire repo. Defines project overview, coding conventions, module boundaries, naming rules, and which commands to run (pnpm dev, drizzle-kit generate, etc.). Claude reads this automatically at the start of every session — it is the single source of truth for how the project is structured and how Claude should behave.
docs/ARCHITECTURE.md	Living architecture reference: module map, data flow diagrams, AWS service relationships, and ADRs (Architecture Decision Records). When Claude is asked to add a new module or change infrastructure, it reads this file first to understand boundaries and avoid introducing inconsistencies.
docs/PRD.md	Product requirements, user stories, and acceptance criteria. Claude references this when implementing features to ensure what is built matches what was specified — without requiring the developer to re-paste requirements into every session.
docs/PROCESS.md	Development workflow: branch naming, PR conventions, migration steps, deployment checklist, and how to use worktrees for parallel feature work. Claude follows this process when creating branches, opening PRs, and coordinating subagent tasks.
apps/*/CLAUDE.md	Package-level context files scoped to mobile or backend. Contains app-specific conventions, screen structure, API patterns, and local gotchas that would clutter the root CLAUDE.md. Subagents spawned for a specific package read the local CLAUDE.md for their scoped instructions.

1.1b GitHub Integration

Claude Code integrates directly with GitHub via the GitHub MCP server, allowing it to manage the full development lifecycle without leaving the terminal. Rather than copying and pasting PR descriptions or manually triaging issues, Claude reads GitHub state and acts on it as part of the same session in which it writes code.

Typical Claude Code + GitHub workflow: Claude implements a feature, runs the affected test suite via Turborepo, commits with a conventional commit message, pushes the branch, opens a PR with a generated description referencing the relevant issue, and requests a review — all in one session. On subsequent sessions, Claude can read PR review comments and address them directly.

GitHub Action	What Claude does
Read issues & milestones	Fetches open issues to understand what to work on next; reads acceptance criteria from issue body to guide implementation scope
Create & push branches	Creates a correctly named branch (e.g. feat/booking-availability-calendar), implements the feature, commits with conventional commit messages, and pushes — all without leaving the session
Open PRs with context	Generates a PR description documenting what changed, which issue it closes (Closes #42), and any migration steps or deployment notes the reviewer needs
Address review comments	Reads inline PR review comments via GitHub MCP, applies the requested changes to the relevant files, and pushes an update — closing the review loop without the developer manually copying feedback into a new session

1.1c MCP Servers — Playwright & Figma

MCP (Model Context Protocol) servers extend Claude Code with specialised tool capabilities. Two MCP servers are integrated into the NannyMom agentic workflow: Playwright for end-to-end browser testing and Figma for design-to-code translation.

MCP Server	Tool	How Claude uses it in NannyMom
Playwright MCP	E2E browser testing	After implementing a backend endpoint or a React Native screen, Claude launches a headless browser via Playwright MCP to run E2E tests against the local dev environment — navigating flows such as booking creation, payment checkout, and care log viewing. Test failures are read back into the session and fixed immediately. This closes the code-test loop without requiring a separate test runner command or human intervention.
Figma MCP	Design-to-code translation	Claude reads a Figma frame directly via Figma MCP — extracting layout, spacing, colours, typography, and component names — and translates the design into React Native screen code in apps/mobile. Tokens (colours, spacing) are synced to packages/ui so the design system stays consistent across all screens. No screenshot hand-off or manual spec reading required.

1.1d Subagents & Git Worktrees

For larger features that span multiple independent packages — for example, a new module that touches the DB schema, backend API, and mobile screen simultaneously — Claude Code can spawn subagents and distribute work across Git worktrees. Each subagent operates in its own worktree (a separate checkout of the same repo at a different path), so multiple workstreams run in parallel without interfering with each other.

Pattern	How it works
Git worktrees for parallel work	git worktree add ../nannymom-feat-booking feat/booking-module creates a second checkout of the repo at a sibling path. Claude spawns a subagent scoped to that worktree to implement the backend booking module while the main session continues working on unrelated mobile UI — no branch conflicts, no stash gymnastics.

Pattern	How it works
Subagent task scoping	The orchestrating agent breaks a large task into independent subtasks and dispatches each to a subagent with a scoped instruction set and the path to the relevant worktree. Each subagent reads the local CLAUDE.md for that package, completes its work, commits, and reports back. The orchestrator then merges results and resolves any shared type conflicts in packages/types.
Example: full-stack feature in parallel	To build the Care Log feature: Subagent A (worktree: feat/care-log-backend) writes the DB migration, care-log module endpoints, and unit tests. Subagent B (worktree: feat/care-log-mobile) reads the Figma MCP frames and builds the React Native screens. The orchestrator waits for both to complete, then runs tsc --build across the monorepo to validate cross-package type consistency before opening PRs.
Why this matters for NannyMom	With 12 backend modules and a full React Native app, a single developer would normally spend days context-switching between layers to ship a feature. Subagents + worktrees compress that into parallel workstreams — effectively multiplying developer throughput without adding headcount. The monorepo structure is what makes this possible: shared types, a single tsconfig, and Turborepo's dependency graph keep all subagents aligned on the same codebase state.

1.2 Smart CI — Turborepo Affected

Turborepo tracks the dependency graph between packages. On every PR it runs only the lint, typecheck, and test pipelines for packages affected by that change — skipping everything else. Results are cached remotely so unchanged packages never rebuild, keeping CI fast regardless of codebase size.

1.3 Unit Test Coverage — All Modules

Test cases for every backend module and shared package are implemented by Claude as part of each feature delivery. Tests run automatically on every PR via Turborepo's affected pipeline.

1.4 Shared TypeScript Configuration

A single base tsconfig is extended by every app and package in the repo — mobile, backend, and infra. Strict mode is enforced uniformly. TypeScript project references give the compiler full visibility of the cross-package dependency graph, catching type errors across boundaries at compile time.

2. AWS API Gateway — Configuration & Route Map

AWS API Gateway (HTTP API type) is the single entry point for all traffic. It is fully managed by AWS — no gateway pods to operate, native integration with CloudWatch, IAM, and private EKS/ECS services via VPC Link.

> Why AWS API Gateway: Zero operational overhead, native CloudWatch metrics and access logging, pay-per-request pricing (~$1/million requests), built-in throttling and WAF integration, and seamless VPC Link to reach private backend containers without exposing them publicly.

2.1 JWT Authorizer (Built-in — No Lambda)

API Gateway's native JWT authorizer validates every inbound request before it reaches the backend — checking the RS256 signature, token expiry, audience (nannymom-api), and issuer (auth.nannymom.com). No Lambda function is needed. Tokens are short-lived (15 minutes per PRD NFR) and issued by the auth module. Role-based access control (mother / nanny / admin) is enforced inside each module, not at the gateway level.

2.2 VPC Link — Private Backend Connectivity

API Gateway never communicates with backend containers over the public internet. All traffic flows through a VPC Link v2 into a private Application Load Balancer inside the VPC, which routes to the backend containers in private subnets. The containers are never directly exposed — only the API Gateway endpoint is public.

2.3 WebSocket API (Separate from HTTP API)

AWS API Gateway provides a managed WebSocket API for all real-time features: live care log updates, mother-nanny chat, presence, and WebRTC signaling. The WebSocket API connects to the ws-hub (Socket.io) service via the same VPC Link. Chat messages are pushed to both parties instantly over the existing connection and persisted to PostgreSQL — no extra service or infrastructure needed.

WebSocket Route	Handler (ws-hub)	Purpose
$connect	Connection auth middleware (JWT from query param)	Authenticate and register socket connection; associate user to their active booking rooms
$disconnect	Cleanup handler	Remove socket from rooms, update presence, mark user offline
$default	Socket.io message router	All Socket.io events routed through here
care_log_subscribe	Join room booking:{id}:mother	Mother subscribes to live care log updates for an active booking
care_log_entry	Care log push handler	Nanny creates a log entry → pushed instantly to mother's screen
chat_message	Chat handler → PostgreSQL + push	Mother or nanny sends a message → persisted to DB → pushed to the other party in real time
chat_read	Read receipt handler	Mark messages as read → push read receipt to sender
video_signal	WebRTC signaling handler	offer / answer / ICE-candidate exchange (only if mediasoup chosen for video)
presence_ping	Presence service	Update online status — Redis TTL if available, otherwise PostgreSQL last_seen_at timestamp

Chat persistence: messages are stored in a PostgreSQL messages table (booking_id, sender_id, body, created_at, read_at). On opening a conversation, the mobile app loads history via a standard HTTP GET /bookings/{id}/messages endpoint. The WebSocket connection handles new messages only — no need to poll.

2.4 Throttling & Cost

Metric	Value	Notes
Default account throttle	10,000 req/s (burst: 5,000)	AWS default; can be increased via support ticket
Per-route throttle	50–300 req/s (see route map above)	Protects individual services from spikes
HTTP API pricing	$1.00 / 1M requests (first 300M/month)	Then $0.90/M; very low at NannyMom scale
WebSocket pricing	$1.00 / 1M messages + $0.25 / 1M connection-minutes	Care log messages: ~$5-20/month at 50K MAU
VPC Link pricing	$0.01/hour per VPC Link (~$7.20/month)	Fixed cost; shared across all routes
Estimated total (50K MAU)	~$30–$60/month	Replaces ~$0 Kong OSS but saves EKS pod costs

3. Paymob Integration — Full Technical Spec

Paymob is the payment gateway of choice for MENA markets (Egypt, Saudi Arabia, UAE, Pakistan). Unlike Stripe's single-step payment intent model, Paymob uses a three-step server-side flow: authentication token → order registration → payment key. The React Native client embeds the Paymob SDK or iFrame using the payment key.

> Paymob supports: Visa/Mastercard, Mobile Wallets (Vodafone Cash, Etisalat Cash, Orange Money, WE Pay), Meeza (Egyptian national card), Fawry (cash payments at kiosks), Bank Installments (BNI, CIB). This covers the full MENA payment landscape that Stripe does not support.

4.1 Payment Flow — Step by Step

Step	Actor	Action	Endpoint / SDK Call	Notes
1	wallet-service	Authenticate with Paymob to get auth token (valid 1 hour; cached in Redis)	POST https://accept.paymob.com/api/auth/tokens	PAYMOB_API_KEY from AWS Secrets Manager. Token cached in Redis with TTL=3500s.
2	wallet-service	Register an order with Paymob (creates order ID)	POST https://accept.paymob.com/api/ecommerce/orders	Body: amount_cents, currency (EGP), items[]. Returns: Paymob order_id.
3	wallet-service	Generate a payment key for the specific integration (card / wallet)	POST https://accept.paymob.com/api/acceptance/payment_keys	Body: auth_token, amount_cents, currency, order_id, integration_id, billing_data. Returns: payment_key (JWT, 1hr TTL).
4	apps/mobile	Embed Paymob SDK / iFrame using the payment_key. User completes payment on Paymob's hosted UI.	Paymob React Native SDK or WebView iFrame	Card data never touches NannyMom servers. PCI handled by Paymob.
5	Paymob	On payment completion, Paymob POSTs a transaction callback (webhook) to NannyMom	POST /webhooks/paymob (booking-service)	HMAC-SHA512 signature verified using PAYMOB_HMAC_SECRET.
6	booking-service	Verify HMAC, check transaction	Internal DB update	Only transition if success=true AND amount matches booking total.

Step	Actor	Action	Endpoint / SDK Call	Notes
		success, update booking status to 'confirmed'		
7	notification-service	Send push + email confirmation to mother and nanny	FCM + SES	Triggered via Redis Pub/Sub 'booking.confirmed' event.

3.2 packages/paymob — SDK Wrapper

The Paymob integration is encapsulated in a shared TypeScript package (packages/paymob) consumed by the backend wallet and booking modules. It handles the three-step Paymob flow: obtaining and caching an auth token (valid ~58 minutes), registering an order, and generating a payment key for the mobile SDK. The token is cached in Redis if available, or in process memory if Redis is not deployed. Incoming webhooks are verified using HMAC-SHA512 before any booking state is updated.

3.3 Paymob Integration IDs per Payment Method

Payment Method	Paymob Integration	Env Variable	Notes
Visa / Mastercard (Card)	Card Integration ID	PAYMOB_CARD_INTEGRATION_ID	Supports 3D Secure; iframe or SDK
Vodafone Cash	Mobile Wallet Integration ID	PAYMOB_WALLET_INTEGRATION_ID	OTP sent to user's registered mobile
Fawry (Cash at kiosk)	Fawry Integration ID	PAYMOB_FAWRY_INTEGRATION_ID	Returns Fawry reference code; async confirmation
Meeza (National Card)	Meeza Integration ID	PAYMOB_MEEZA_INTEGRATION_ID	Egyptian national debit card
Bank Installments	Installments Integration ID	PAYMOB_INSTALLMENTS_INTEGRATION_ID	Selected banks only; shown based on BIN lookup

3.4 In-App Wallet & Paymob

The NannyMom in-app wallet is separate from Paymob. Promotional credits and refunds live in a wallet ledger in PostgreSQL. At checkout, the wallet balance is deducted first and the remainder is charged via Paymob — a hybrid flow coordinated by the booking module. If Paymob payment succeeds (confirmed via webhook), the wallet hold is committed. If payment fails, the hold is released and the booking remains pending.

4. AWS Infrastructure — Complete Service Map

All infrastructure is defined as code using AWS CDK (Cloud Development Kit) in TypeScript, living in the infra/ directory of the monorepo. CDK synthesizes to CloudFormation and is deployed via GitHub Actions. The AWS account structure uses three accounts: dev, staging, and production, all managed under an AWS Organizations root.

> Why AWS CDK over Terraform: The entire NannyMom codebase is TypeScript — mobile, backend, shared packages, and now infrastructure. CDK keeps that consistency: one language, one repo, one context for Claude Code to reason across. You get type-safe infrastructure (autocomplete, compile-time errors on misconfigured resources), native AWS constructs (no HCL to learn), and reusable CDK Constructs that encode your patterns once and apply them everywhere. Terraform is the right choice when you need multi-cloud portability or a dedicated platform team — neither applies here. CDK is the natural fit for a solo TypeScript developer building on AWS.

4.1 AWS Services Used

Note: Compute and database options are presented as stakeholder decisions in Sections 5.2 and 5.3 respectively. All other AWS services are fixed regardless of those choices.

AWS Service	Purpose	Config/Tier	Monthly Cost (50K MAU)
Compute — see Section 5.2	App Runner OR ECS Fargate — stakeholder decision	See Section 5.2 for full comparison	**See Section 5.2**
Database — see Section 5.3	Classic RDS PostgreSQL OR Aurora — stakeholder decision	See Section 5.3 for full comparison	**See Section 5.3**
Amazon ElastiCache (Redis) — OPTIONAL	Paymob token caching, presence TTL, Socket.io multi-instance adapter, rate limiting — all replaceable without Redis at v1.0	cache.r6g.large — only needed if Redis path chosen	~$380/month (optional — see Section 4.2)
PostgreSQL Full-Text + PostGIS	Nanny search (geo + skill matching), marketplace search, FAQ — handled natively in Aurora/RDS using tsvector and PostGIS extensions	Built into the database — no extra service	$0 (included in DB cost)
Amazon S3	Video clips, care log photos, marketplace images, CDK bootstrap bucket	Standard tier; lifecycle rules for 7-day clip deletion	~$60/month
Amazon CloudFront — OPTIONAL	CDN for S3 media delivery (nanny photos, listing images, video clips). Without CloudFront, media is served directly from S3 presigned URLs — viable at v1.0 but higher latency for users outside the bucket region and 10× higher S3 data transfer costs at scale.	Global PoP; signed URLs for video clips — only needed if CDN path chosen	~$120/month (optional — see Section 4.2a)

AWS Service	Purpose	Config/Tier	Monthly Cost (50K MAU)
AWS Secrets Manager	All secrets: Paymob keys, DB credentials, JWT private key	~15 secrets; rotated quarterly	~$12/month
Amazon SES	Transactional emails (booking confirmation, ticket response)	Sandbox → Production; SES domain verification	~$40/month
Amazon SNS	Emergency SMS alerts to on-call support agents	SNS SMS (Egypt: ~$0.02/SMS)	~$15/month
Amazon ECR	Private Docker image registry for all service containers	Lifecycle: keep last 10 images	~$10/month
Amazon Route 53	DNS: api.nannymom.com, ws.nannymom.com, admin.nannymom.com	Hosted zone + health checks	~$10/month
AWS Certificate Manager	TLS certs for all domains (auto-renewal)	Free with Route 53 / ALB	$0
Amazon CloudWatch	Logs aggregation; container logs; billing alerts	Log retention: 30 days	~$80/month
Image Moderation	Manual review queue + user reporting — content flagged by users is reviewed via the admin module	No external service needed at v1.0	$0
AWS API Gateway	HTTP API + WebSocket API — single managed entry point	HTTP API (pay-per-request); WS API for real-time	~$30–$60/month
AWS WAF	Web Application Firewall attached to API Gateway	Managed rule groups (SQLi, XSS, rate-based rules)	~$30/month
Amazon VPC	Private networking; NAT Gateway for outbound; VPC endpoints	1 VPC, 3 AZs, private + public subnets	~$90/month (NAT GW)
TOTAL (excl. compute + DB + Redis)			**~$470–$520/month fixed · +$380 if Redis**

4.2 ElastiCache Redis — Optional Service

Redis is not required for v1.0. Every use case Redis would cover has a viable alternative using services already in the stack. The decision comes down to operational simplicity vs. performance ceiling.

Use case	With Redis	Without Redis (v1.0 alternative)
Paymob auth token caching	Token cached in Redis, shared across all backend instances — zero duplicate auth calls	Token cached in process memory per instance — negligible extra Paymob auth calls if multiple instances run; fully acceptable at v1.0
Presence (nanny/mother online)	Redis key with TTL — instant lookup, auto-expires on disconnect	PostgreSQL last_seen_at timestamp — consider online if updated within last 30s; slightly higher DB read load but negligible
Socket.io multi-instance room state	Redis adapter shares room membership across multiple ws-hub instances seamlessly	Not needed if ws-hub runs as a single instance — Socket.io manages rooms in process memory by default

Use case	With Redis	Without Redis (v1.0 alternative)
Rate limiting (per-user)	Redis sliding window counters — accurate across all instances	API Gateway stage-level throttling — sufficient for v1.0; per-user granularity can be added later
Pub/Sub (event fanout)	Redis Pub/Sub broadcasts events (e.g. booking.confirmed) to all backend instances	Direct in-process event emitter — works fine with a single modular monolith container

	With Redis	Without Redis
Monthly cost	~$380/month (ElastiCache r6g.large)	$0
Operational overhead	One more managed service to monitor, back up, and size	None
Scale ceiling	Handles horizontal scaling of all services cleanly	Requires Redis before scaling ws-hub beyond 1 instance
Local dev	Redis container added to docker-compose	One less service in docker-compose
Recommended for	50K+ MAU or when running multiple ws-hub instances	v1.0 launch — add Redis when horizontal scaling is needed

Recommendation: launch without Redis. The only hard dependency is the Socket.io multi-instance adapter — if you ever scale ws-hub to two or more instances, add ElastiCache Redis at that point. All other use cases (Paymob caching, presence, rate limiting) work without it at v1.0 scale.

4.2a Amazon CloudFront — Optional Service

CloudFront is not required for v1.0. Media files (nanny profile photos, care log photos, marketplace images, video clips) can be served directly from S3 using presigned URLs. The trade-off is latency and cost at scale.

	With CloudFront	Without CloudFront (S3 direct)
Media delivery latency	Low — served from nearest AWS edge PoP globally	Higher — served from the S3 bucket region (eu-south-1 or me-south-1 for Egypt)
Video clip security	CloudFront signed URLs — time-limited, tamper-proof	S3 presigned URLs — also time-limited but generated per request by the backend
S3 data transfer cost	~$0.0085/GB (CloudFront origin transfer)	~$0.09/GB (S3 internet egress) — ~10× more expensive at high volume
Monthly cost at 50K MAU	~$120/month	~$0 CDN cost; S3 egress grows with usage
Operational overhead	One CloudFront distribution to configure and invalidate on deploys	None — S3 presigned URLs generated on demand
Cache invalidation	Required on profile photo updates — CDK handles this	Not needed — presigned URLs always point to latest object

Recommendation: launch without CloudFront. At v1.0 with 10K MAU, S3 egress cost is negligible (~$5–10/month). Add CloudFront when media volume grows or when users outside Egypt report slow load times. The

CDK construct is straightforward to add later — it does not require any backend code changes, only a DNS update.

4.3 Compute Options — Stakeholder Decision

The backend runs three containers: the modular monolith (apps/backend), the WebSocket hub (ws-hub), and the video service (mediasoup). Two AWS compute options are presented below. All other architecture decisions are identical regardless of which is chosen.

The three containers to run: (1) nannymom-backend — modular monolith · (2) nannymom-ws-hub — Socket.io · (3) nannymom-video — mediasoup SFU

Option A — AWS App Runner

App Runner is AWS's fully managed container platform. You provide a Docker image from ECR; AWS handles provisioning, scaling, load balancing, TLS, and health checks with zero configuration. No VPC setup required for the service itself.

Attribute	Detail
How it works	Point App Runner at an ECR image. AWS auto-provisions compute, configures an HTTPS endpoint, and scales containers up/down based on incoming requests. No servers, no clusters, no networking config.
Scaling	Automatic, request-based. Scales to zero when idle (cost saving). Scales up within seconds on traffic spike. Min/max instance count configurable.
VPC access	App Runner supports VPC connectors to reach private RDS Aurora and ElastiCache. Defined as a CDK Construct in infra/lib/constructs/SecureService.ts — one reusable pattern for all three containers.
WebSocket support	App Runner supports persistent HTTP/2 connections but WebSocket support is limited — the ws-hub may require ECS Fargate instead (hybrid approach possible).
Video service	mediasoup requires UDP ports for WebRTC which App Runner does not expose. The video service must run on ECS Fargate or EC2 regardless of this choice.
Operational overhead	Minimal — no cluster management, no task definitions, no load balancer config. Deploy by pushing a new image to ECR and triggering a redeployment.
Observability	Built-in CloudWatch metrics and logs. Less granular than ECS but sufficient for most needs.
Pricing model	Pay per vCPU-second and GB-second of active compute. Scales to near-zero cost during off-peak hours. No idle cost when scaled to zero.
Best for	Single developer with agentic development — minimal ops overhead means more time on product features. Deploy by pushing an image; AWS handles the rest.
Limitations	No UDP (blocks mediasoup), limited WebSocket support, less control over networking, no GPU instances, less mature than ECS.

Cost Component	10K MAU / month	50K MAU / month	Notes
nannymom-backend (App Runner)	~$60–$120	~$200–$400	1 vCPU / 2GB base; scales with traffic

Cost Component	10K MAU / month	50K MAU / month	Notes
nannymom-ws-hub	~$40–$80	~$120–$250	Persistent connections; may need ECS Fargate
nannymom-video (ECS Fargate)	~$80–$150	~$250–$500	Must be Fargate — App Runner lacks UDP
Option A Total (compute only)		**~$570–$1,150/month**	Add ~$1,650 fixed AWS services = ~$2,200–$2,850 total

Option B — AWS ECS with Fargate

ECS (Elastic Container Service) is AWS's native container orchestrator. Fargate is the serverless compute engine for ECS — you define CPU and memory per task, and AWS runs the container without you managing any EC2 instances or clusters.

Attribute	Detail
How it works	You define an ECS Task Definition (CPU, memory, image, env vars, ports) and an ECS Service (desired count, auto-scaling policy, ALB target group). Fargate provisions the underlying compute invisibly.
Scaling	Target tracking auto-scaling on CPU %, memory %, or custom CloudWatch metrics (e.g. active WebSocket connections). Min/max task count configured per service.
VPC access	Full VPC integration — tasks run inside your private subnets with direct access to RDS Aurora and ElastiCache with no VPC connector needed.
WebSocket support	Full support. ws-hub runs as a standard ECS Fargate service behind an ALB with sticky sessions enabled (required for Socket.io).
Video service	Full UDP port support. mediasoup runs natively on ECS Fargate with the required port ranges (10000-10100/UDP) opened in the security group.
Operational overhead	Low-to-medium. No EC2 nodes to manage, but requires understanding of task definitions, service configuration, ALB target groups, and IAM task roles.
Observability	Deep CloudWatch integration — Container Insights gives per-task CPU/memory metrics, log streaming to CloudWatch Logs, and X-Ray tracing support.
Pricing model	Pay per vCPU-hour and GB-hour of allocated (not used) resources. No scale-to-zero — minimum cost even when idle. Reserved capacity available for discount.
Best for	Single developer comfortable with AWS fundamentals — more config upfront but full control over networking, WebSocket, and UDP. Claude Code can manage CDK stacks in TypeScript alongside the rest of the codebase.
Limitations	More setup than App Runner, always-on cost (no scale-to-zero), requires ALB configuration, IAM task role setup.

Cost Component	10K MAU / month	50K MAU / month	Notes
nannymom-backend (Fargate)	~$80–$140	~$240–$480	0.5 vCPU / 1GB × 2 tasks base; scales to 8 tasks
nannymom-ws-hub (Fargate)	~$60–$100	~$180–$320	0.5 vCPU / 1GB; ALB sticky sessions
nannymom-video (Fargate)	~$100–$180	~$300–$600	1 vCPU / 2GB; UDP ports open; c-series preferred

Cost Component	10K MAU / month	50K MAU / month	Notes
Application Load Balancer	~$20	~$35	Shared across all three services
Option B Total (compute only)		**~$755–$1,435/month**	Add ~$1,650 fixed AWS services = ~$2,400–$3,135 total

4.4 Video Streaming — Stakeholder Decision

Live nanny monitoring is a core PRD feature requiring video from the nanny's device to the mother's screen. The PRD specifies under 2 seconds latency. Three options exist — two fully managed AWS services and the self-hosted WebRTC approach currently in the architecture.

Option A — Amazon IVS Real-Time Streaming

Attribute	Detail
What it is	AWS's fully managed video service built on Twitch technology. Real-Time Streaming uses WebRTC under the hood and delivers video via AWS's global infrastructure with no servers to manage.
Latency	Under 300ms host-to-viewer (Real-Time Streaming tier). Meets the PRD <2s requirement with significant headroom.
Architecture impact	No mediasoup container, no coturn TURN server, no UDP ports. Eliminates the constraint that forces ECS Fargate — App Runner becomes viable for all three services.
Session recording	Low-Latency Streaming tier only (HLS-based, 3–5s latency). Real-Time Streaming cannot directly record to S3 — you must route through a Low-Latency channel first to enable recording.
Scale	Up to 12 hosts and 10,000 viewers per stage. More than sufficient for NannyMom 1:1 sessions.
Ops overhead	Zero — fully managed by AWS. No infrastructure to provision, scale, or maintain.
SDK	iOS, Android, and Web broadcast SDKs provided. Integrates with React Native via the native modules.
Cost model	Pay per participant-hour connected to a stage resource. No fixed infra cost.
Best for	Solo developer who wants zero video infrastructure. Removes the ECS Fargate constraint and simplifies the entire compute decision.

Cost Component	10K MAU	50K MAU	100K MAU
Participant-hours (host + viewer, avg 1hr/day active sessions)	~$50–$100/month	~$250–$500/month	~$500–$1,000/month
S3 storage for session recordings	~$10/month	~$30/month	~$60/month
Option A Total	**~$60–$110/month**	**~$280–$530/month**	**~$560–$1,060/month**

Option B — Amazon Kinesis Video Streams (WebRTC)

Attribute	Detail
What it is	AWS's managed WebRTC signaling and TURN infrastructure. Handles ICE/STUN/TURN so peer-to-peer connections work across NAT. Media flows peer-to-peer when possible; TURN relay when not.
Latency	Sub-second peer-to-peer when direct connection is established. TURN relay adds 50–200ms. Meets <2s PRD requirement.
Architecture impact	Replaces coturn (no EC2 TURN server needed) but still requires mediasoup or direct P2P — KVS handles signaling and TURN only, not media forwarding. If 1:1 only, media can go P2P with no SFU. If recording is needed, mediasoup or another mechanism is still required.
Session recording	Not built-in for WebRTC path. Requires a separate recording mechanism (mediasoup server-side recording → S3, or a recording Lambda).
Scale	Designed for IoT camera use cases (baby monitors, doorbells). Works for 1:1 sessions; multi-viewer requires cloud forwarding.
Ops overhead	Low — signaling and TURN are managed. Still need to handle media routing logic in the app.
Cost model	Per active signaling channel/month ($0.03), per million signaling messages ($2.25), per 1,000 TURN minutes ($0.12).
Best for	Teams already using AWS IoT or wanting P2P without a separate TURN server EC2. Less turnkey than IVS for a consumer app.

Cost Component	10K MAU	50K MAU	100K MAU
Active signaling channels ($0.03/channel/month)	~$15/month	~$75/month	~$150/month
Signaling messages ($2.25/million)	~$5/month	~$25/month	~$50/month
TURN minutes ($0.12/1,000 min — 40% of sessions)	~$20/month	~$100/month	~$200/month
Option B Total	**~$40/month**	**~$200/month**	**~$400/month**

Option C — Self-Hosted mediasoup SFU (Current Architecture)

Attribute	Detail
What it is	Open-source WebRTC SFU (Selective Forwarding Unit) running as a Docker container on ECS Fargate. Receives the nanny's video stream and forwards it to the mother without re-encoding, keeping CPU usage low.
Latency	Sub-100ms end-to-end in ideal conditions. Best raw latency of the three options.
Architecture impact	Requires ECS Fargate (UDP ports). Forces ECS Fargate as the compute choice — App Runner is not viable. Requires self-managed coturn TURN server on EC2.
Session recording	Native server-side recording built into mediasoup. Records directly to S3. Full control over clip format and retention.
Scale	Fully controllable — scale Fargate tasks horizontally. Can handle many concurrent sessions with proper task sizing.
Ops overhead	High relative to managed options. mediasoup config, coturn config, UDP security group rules, container scaling tuning, TURN secret rotation.
Cost model	Fixed ECS Fargate task cost (always-on, no scale-to-zero) + EC2 coturn instance. Predictable but higher floor cost.

Attribute	Detail
Best for	Teams needing maximum control over video quality, recording format, and latency. Worth the ops cost at scale — not ideal for v1.0 with a solo developer.

Cost Component	10K MAU	50K MAU	100K MAU
ECS Fargate (mediasoup container, always-on)	~$80–$150/month	~$250–$500/month	~$500–$1,000/month
EC2 coturn TURN server (t3.medium, Elastic IP)	~$40/month	~$80/month	~$160/month
Option C Total	**~$120–$190/month**	**~$330–$580/month**	**~$660–$1,160/month**

4.4a Video Decision Matrix

Criterion	Option A — IVS Real-Time	Option B — KVS WebRTC	Option C — mediasoup	Weight for NannyMom
Latency	<300ms ✓	<500ms ✓	<100ms ✓	High — PRD requires <2s (all pass)
Ops overhead	Zero ✓	Low ✓	High — coturn + SFU config	Critical — solo developer
Compute constraint	None — App Runner viable ✓	Still needs SFU for recording	Forces ECS Fargate ✗	High — affects whole architecture
Session recording	Via Low-Latency channel (extra step)	Custom solution needed	Native, direct to S3 ✓	High — care session clips are PRD feature
React Native SDK	Official AWS SDK ✓	Community/custom integration	Custom WebRTC integration	High — mobile-first app
Cost at 50K MAU	~$280–$530/month	~$200/month ✓	~$330–$580/month	Medium
Cost at scale (100K+)	Scales with usage	Lowest cost ✓	Predictable fixed cost	Medium
All-AWS stack	Yes ✓	Yes ✓	Yes ✓	Medium
Production readiness	Fully managed, battle-tested ✓	Managed signaling, custom media	Self-managed, full control	High
VERDICT	**Best if: simplicity is priority, willing to accept the recording workaround, want App Runner as compute option**	**Best if: lowest cost at scale, comfortable building custom recording, 1:1 P2P sessions only**	**Best if: maximum control needed, recording format matters, planning for scale post-launch**	

Key dependency: If Option A (IVS) is chosen for video, App Runner becomes viable for all three containers — removing the ECS Fargate constraint entirely. If Option B (KVS) or Option C (mediasoup) is chosen, ECS Fargate remains required for the video service. This makes the video decision upstream of the compute decision.

4.5 Database Options — Stakeholder Decision

Both options are fully PostgreSQL-compatible — application code, queries, and the Drizzle ORM schema in packages/db work identically with either. The difference is entirely in how AWS manages the underlying infrastructure, how it scales, and what it costs.

Option A — Classic RDS PostgreSQL (Multi-AZ)

Attribute	Detail
What it is	Standard PostgreSQL on a managed AWS EC2 instance. AWS handles backups, patching, and Multi-AZ failover. Underneath it is plain Postgres on an EBS volume.
Failover time	60–120 seconds. AWS detects primary failure, promotes standby, updates DNS. App receives connection errors during this window.
Storage scaling	Manual — you pre-provision (e.g. 100GB) and resize when needed. Resize is online but requires planning ahead.
Read replicas	Up to 5. Replication via WAL streaming — lag can reach seconds to minutes under write load. Not suitable for balance or booking queries; acceptable for search.
Vertical scaling	Requires a reboot with ~1–3 min downtime (Multi-AZ failover). Must be done manually when CPU/memory pressure appears.
I/O pricing	Included in instance price — no surprise bills.
Best for	Solo developer who wants predictable monthly billing and zero database surprises. No scaling decisions to make until 100K+ MAU.

Cost Component	10K MAU	50K MAU	100K MAU
Instance (db.r6g.large, Multi-AZ)	~$350/month	~$350/month	~$700/month (upgrade to xlarge)
Storage (100–300GB)	~$12/month	~$23/month	~$46/month
Backup retention (35 days)	~$10/month	~$19/month	~$38/month
Option A Total	**~$372/month**	**~$392/month**	**~$784/month**

Option B — Aurora PostgreSQL Serverless v2

Attribute	Detail
What it is	AWS's cloud-native PostgreSQL-compatible engine with a serverless compute layer. Scales write and read capacity automatically in sub-second increments based on actual load. No instance resizing, no reboots.
Failover time	20–30 seconds (often under 10s). Replicas share the same distributed storage layer — no data to catch up on.

Attribute	Detail
Storage scaling	Fully automatic in 10GB increments up to 128TB. Zero intervention required, ever.
Read replicas	Up to 15. Replicas share storage — lag is consistently under 100ms. Replicas also auto-scale based on load policies.
Vertical scaling	Automatic — Serverless v2 scales ACUs (compute) up and down within your configured min/max range with no downtime and no human action.
I/O pricing	Standard: $0.20 per 1M I/O requests (can surprise at high volume). I/O-Optimized: flat 43% higher instance price, no per-I/O charge — switch any time.
Best for	Solo developer who wants zero database operations — storage, scaling, and failover are fully automated. CDK stack in infra/lib/stacks/database-stack.ts is the only config needed; no manual DB intervention required.

Cost Component	10K MAU	50K MAU	100K MAU
ACU-hours (min 0.5, max 16 ACU)	~$80–$200/month	~$200–$450/month	~$400–$800/month
Storage (auto-scaling)	~$10/month	~$20/month	~$40/month
I/O (standard tier)	~$20–$50/month	~$100–$200/month	~$200–$400/month
Backup retention (35 days)	~$2/month	~$4/month	~$8/month
Option B Total	**~$112–$252/month**	**~$324–$674/month**	**~$648–$1,248/month**

4.5a Database Decision Matrix

Criterion	Option A — Classic RDS PG	Option B — Aurora Serverless v2	Weight for NannyMom
Failover speed	60–120 seconds	20–30 seconds ✓	High — 99.9% SLA requirement
Storage management	Manual provisioning	Fully automatic ✓	High — startup convenience
Scaling under traffic spike	Manual vertical resize + reboot	Automatic, no downtime ✓	High — unpredictable growth
Read replica lag	Seconds to minutes	< 100ms ✓	Medium — nanny search queries
Cost at 10K MAU	~$372/month ✓	~$112–$252/month ✓	High — Aurora cheaper at low load
Cost at 50K MAU	~$392/month ✓	~$324–$674/month	High — RDS more predictable at scale
Cost at 100K MAU	~$784/month ✓	~$648–$1,248/month	High — RDS wins at sustained load

Criterion	Option A — Classic RDS PG	Option B — Aurora Serverless v2	Weight for NannyMom
I/O cost surprise risk	None ✓	Yes — monitor closely	High — budget planning
Off-peak cost saving	None — always-on	Scales to 0.5 ACU overnight ✓	Medium — Egypt timezone app
Operational overhead	Very low ✓	Near-zero ✓	High
PostgreSQL compatibility	100% ✓	100% ✓	— same code either way
VERDICT	**Best if: predictable cost is priority, comfortable with occasional manual scaling**	**Best if: zero ops is priority, comfortable monitoring I/O costs in CloudWatch**	

Note on Aurora I/O-Optimized: If Option B is chosen and I/O costs grow beyond ~$150/month, switching to Aurora I/O-Optimized pricing (no per-I/O charge, flat +43% on instance) is a one-click change with zero downtime. Monitor monthly I/O in CloudWatch from day one.

4.6 Option Comparison — Decision Matrix

Criterion	Option A — App Runner	Option B — ECS Fargate	Weight
Operational complexity	Very Low ✓	Low–Medium	Critical — 1 developer + agentic workflow
WebSocket (ws-hub)	Limited — may need hybrid	Full support ✓	Critical — real-time care log
WebRTC / UDP (video)	Not supported ✗	Full support ✓	Critical — live monitoring
VPC / private DB access	Via connector (extra config)	Native ✓	High — security requirement
Scale to zero (off-peak)	Yes — cost saving ✓	No — always-on cost	Medium
Cost at 10K MAU	~$2,200–$2,500/month ✓	~$2,400–$2,700/month	Medium
Cost at 50K MAU	~$2,200–$2,850/month ✓	~$2,400–$3,135/month	High
Cost at 100K MAU	~$3,500–$5,000/month	~$3,200–$4,800/month ✓	High — ECS more efficient at scale
AWS knowledge required	Minimal ✓	AWS fundamentals needed	High — single developer context
Deployment simplicity	Push image to ECR → redeploy	Update task definition → rolling deploy	Medium
Production readiness	Good for v1.0	Better for v1.0+ scale	Medium

Criterion	Option A — App Runner	Option B — ECS Fargate	Weight
Hybrid requirement	ws-hub + video need ECS anyway	All three services on one platform ✓	High
VERDICT	**Best if: solo developer, tight timeline, minimal AWS ops desired**	**Best if: comfortable with AWS, full WebSocket + UDP needed on one platform**	

Important constraint: The video service (mediasoup WebRTC) requires UDP ports and cannot run on App Runner under any configuration. If Option A is chosen, the video service must run on ECS Fargate regardless — making Option A a hybrid architecture. If WebRTC video monitoring is a core v1.0 feature, Option B (full ECS Fargate) avoids the complexity of managing two different compute platforms.

4.6 Network Architecture (applies to all options)

All resources run inside a single AWS VPC spanning three Availability Zones. Public subnets hold the Application Load Balancer, NAT Gateways for outbound container traffic, and the coturn TURN server (Elastic IP) for WebRTC NAT traversal. Private subnets hold all compute (App Runner connector or ECS Fargate tasks), RDS/Aurora, and ElastiCache Redis — none of these are reachable from the public internet. VPC Endpoints for S3, ECR, Secrets Manager, and CloudWatch Logs ensure AWS service traffic stays on the AWS backbone without traversing the internet.

4.7 AWS Secrets Manager — All Secrets

Secret Name	Value	Consumed By	Rotation
nannymom/prod/db	PostgreSQL connection string (Aurora endpoint)	All services via @nannymom/db	Quarterly (automated)
nannymom/prod/redis	ElastiCache Redis connection URL	All services, ws-hub	On change
nannymom/prod/jwt-private-key	RS256 private key (PEM)	auth-service (signing)	Annually
nannymom/prod/jwt-public-key	RS256 public key (PEM)	Kong JWT plugin, all services (verify)	With private key
nannymom/prod/paymob-api-key	Paymob API key	packages/paymob → wallet-service	On change
nannymom/prod/paymob-hmac-secret	Paymob HMAC secret for webhook verification	booking-service webhook handler	On change
nannymom/prod/paymob-integration-ids	JSON: {card, wallet, fawry, meeza, installments}	booking-service	On change
nannymom/prod/fcm-service-account	Firebase Admin SDK JSON (base64)	notification-service	Annually
nannymom/prod/ses-smtp	AWS SES SMTP credentials	notification-service	Quarterly
nannymom/prod/twilio	Twilio AccountSID + AuthToken	notification-service, support-service	Quarterly
nannymom/prod/google-maps	Google Maps Platform API key	community-service, nanny-service	Annually

Secret Name	Value	Consumed By	Rotation
nannymom/prod/checkr	Checkr API key for background checks	nanny-service	Annually
nannymom/prod/zendesk	Zendesk API token + subdomain	support-service	Quarterly
nannymom/prod/turn-secret	coturn TURN REST API secret	video-service	Monthly

5. Notification System

The notification module dispatches across three channels — push (FCM), SMS (SNS), and email (SES) — based on the event type and the user's preferences. All three channels are triggered from within the modular monolith; no extra service or queue is required for v1.0.

> FCM (Firebase Cloud Messaging) is a free Google service and the only supported way to deliver push notifications to both iOS and Android. It is an external dependency — not an AWS service — and must be configured in Firebase Console. The FCM server key is stored in AWS Secrets Manager.

5.1 Notification Channels

Channel	Service	Use cases	Cost
Push	Firebase Cloud Messaging (FCM)	Booking requests, booking confirmed, nanny arrived, care log started, new chat message, payment received, booking reminders	Free (no limit)
SMS	Amazon SNS	Phone OTP (auth), critical booking alerts (nanny late, booking cancelled)	~$0.02/SMS in Egypt — ~$15/month
Email	Amazon SES	Booking confirmation + receipt, support ticket response, weekly care summary	~$0.10/1K emails — ~$40/month

5.2 Notification Events — All Triggers

Event	Push	SMS	Email	Recipient
Booking request received	✓	—	—	Nanny
Booking confirmed	✓	✓	✓	Mother + Nanny
Booking cancelled	✓	✓	✓	Mother + Nanny
Nanny on the way	✓	—	—	Mother
Nanny arrived	✓	—	—	Mother
Care log started	✓	—	—	Mother
New care log entry	✓	—	—	Mother
New chat message	✓	—	—	Recipient
Payment received	✓	—	✓	Nanny
Payment failed	✓	✓	—	Mother
Booking reminder (1hr before)	✓	—	—	Mother + Nanny
Phone OTP	—	✓	—	User
Support ticket response	✓	—	✓	Mother or Nanny
Weekly care summary	—	—	✓	Mother

5.3 User Notification Preferences

Each user controls their notification preferences per channel. Preferences are stored in PostgreSQL and checked by the notification module before dispatching. Users can disable push, SMS, or email independently per event category — except for OTP SMS which is always sent regardless of preferences.

Preference	Scope	Default
Push notifications	Per category (booking, chat, payments, reminders)	All on
Email notifications	Per category (booking, payments, summaries)	All on
SMS notifications	OTP only (cannot be disabled)	Always on
Quiet hours	Daily time window — push suppressed, queued for morning	Off by default

5.4 Delivery Architecture

For v1.0, the notification module is called directly in-process from the booking, chat, wallet, and auth modules — no queue or Pub/Sub needed. If Redis is added later, notifications can be moved to a Redis Pub/Sub channel or a BullMQ queue for retry handling and decoupling. FCM handles its own retry for push delivery. Failed SES emails and SNS SMS are logged to CloudWatch with an alert threshold for investigation.

6. CI/CD Pipeline — GitHub Actions + AWS

6.1 Pipeline Overview

Three GitHub Actions workflows cover the full delivery lifecycle. On every PR, Turborepo runs lint, typecheck, and tests only for affected packages, and CDK diff shows exactly which AWS resources will change. On merge to main, CDK deploys updated stacks to staging, DB migrations run as a one-off ECS task before the new container version starts, then the backend service is redeployed. Production deploys trigger on a release tag and require manual approval in GitHub.

Trigger	Workflow	Key Steps
Every PR	ci.yml	Turborepo affected lint + typecheck + test; CDK diff; Docker build + push to ECR
Merge to main	deploy-staging.yml	CDK deploy staging stacks; run DB migrations; redeploy backend service
Release tag (v*.*.*)	deploy-production.yml	Manual approval gate; CDK deploy production stacks; run DB migrations; redeploy

6.2 React Native — EAS Build + EAS Update

Trigger	Action	Output	Notes
PR opened	EAS Build (development profile)	APK + IPA for QA device testing	Detox E2E runs against this build
Merge to main	EAS Build (staging profile) + EAS Update	Staging OTA JS bundle pushed to staging channel	Testers on TestFlight / Firebase App Distribution see instant update
Release tag (v*.*.*)	EAS Build (production profile)	Production IPA + AAB submitted to App Store + Play Store	Full store review required; JS-only changes can use EAS Update instead
JS-only change (no native)	EAS Update only	OTA bundle pushed to production channel	Users get update on next app open; no App Store review needed

6.3 Database Migrations in CI/CD

Drizzle ORM migrations run as a one-off ECS Fargate task (defined in the CDK compute stack) before any new container version starts serving traffic. The deploy pipeline waits for the migration task to complete successfully before triggering a service redeployment. If the migration fails, the pipeline halts and the existing container continues running on the previous schema — ensuring no downtime from failed migrations. Migrations are forward-only; rollback is handled by deploying a new corrective migration.

7. Local Development Environment

A single docker-compose.yml at the repo root spins up all infrastructure dependencies. Each service runs natively on the host with pnpm dev (ts-node-dev hot reload). No Docker needed for the services themselves during development — only for infra.

7.1 Local Infrastructure

A single docker-compose.yml at the repo root spins up PostgreSQL and Redis for local development. The backend modules run natively on the host with hot reload — no Docker needed for the services themselves. A local nginx reverse proxy routes all traffic through a single port, mirroring the API Gateway behaviour in production.

7.2 Getting Started

Setup is a three-step process: clone the repo and run pnpm install to install all workspace dependencies, copy .env.example to .env.local and fill in the third-party API keys (Paymob, Google Maps), then start infrastructure with docker compose up and run the DB migration. From there, a single pnpm dev command starts all backend modules in parallel with hot reload and launches the Expo dev server for mobile.

7.3 Expo — iOS Simulator & Build

Expo bare workflow gives full access to the iOS Simulator for development, EAS Build for production binaries, and EAS Update for over-the-air JS bundle updates without App Store review.

Task	Command	Notes
Start Expo dev server	pnpm --filter apps/mobile start	Opens Expo CLI; choose simulator or device from menu
Run on iOS Simulator	Press i in Expo CLI OR pnpm --filter apps/mobile ios	Requires Xcode + iOS Simulator installed on macOS. Launches in simulator automatically.
Run on physical iPhone	Press s → scan QR with Expo Go app	Expo Go supports most features; native modules (WebRTC, SQLite) require dev build
Install dev build on simulator	pnpm --filter apps/mobile run ios --simulator	Uses bare workflow native build; required for WebRTC + WatermelonDB
EAS dev build (simulator)	eas build --profile development --platform ios	Builds .app file for simulator; includes all native modules; ~5 min on EAS cloud
EAS dev build (physical device)	eas build --profile development --platform ios --local OR cloud	Builds IPA installable via TestFlight or direct install with dev certificate
EAS staging build	eas build --profile staging --platform ios	TestFlight distribution; used for stakeholder and user testing
EAS production build	eas build --profile production --platform ios	App Store submission via eas submit --platform ios
OTA update (JS only)	eas update --branch production --message 'fix: ...'	Ships JS bundle update to users; no App Store review; available on next app launch
Preview update on simulator	eas update --branch staging && expo start --update-url ...	Test OTA bundle on local simulator before promoting to production branch

7.4 EAS Build Profiles

Three build profiles are defined in eas.json: development (simulator build with dev client, all native modules enabled), staging (internal distribution to TestFlight / Firebase App Distribution), and production (App Store IPA and Play Store AAB). JS-only changes can be shipped at any time as OTA updates via EAS Update without requiring a new store build or App Store review.

7.5 Paymob in Development

Scenario	Approach	Notes
Test card payments	Use Paymob sandbox environment (accept.paymobsolutions.com)	Different base URL from production; separate API key
Test wallet (Vodafone Cash)	Paymob sandbox provides test phone numbers	Simulates OTP flow without real SIM
Test Fawry	Paymob sandbox returns mock Fawry reference	No real kiosk payment needed
Test webhooks locally	Use ngrok or Cloudflare Tunnel to expose localhost:3003	Set Paymob callback URL to tunnel URL in sandbox dashboard
HMAC verification in dev	Use sandbox HMAC secret from Paymob dashboard	Separate from production secret in .env.local

8. Cost Breakdown — Paymob + AWS Stack

Category	Service	10K MAU	50K MAU	100K MAU	Notes
Compute	App Runner OR ECS Fargate (see Section 4.2)	$200–$400	$570–$1,150	$1,300–$2,400	Depends on compute option chosen
Database	Aurora Serverless v2 OR RDS PG (see Section 4.3)	$112–$392	$324–$674	$648–$1,248	Depends on database option chosen
Cache (optional)	ElastiCache Redis — add when scaling ws-hub beyond 1 instance	$0 or $200	$0 or $380	$0 or $760	Optional — see Section 4.2
Search	PostgreSQL full-text + PostGIS	$0	$0	$0	Built into DB — no extra service
Storage	S3 (required) + CloudFront (optional)	$40 / $80+	$60 / $180+	$120 / $380+	S3 fixed; CloudFront adds ~$120/month at 50K MAU
Video infra	coturn EC2 + mediasoup	$280	$560	$1,120	c5.2xlarge × 2–4
Secrets	AWS Secrets Manager	$5	$12	$20	~15 secrets
Email	AWS SES	$10	$40	$100	$0.10/1K emails
SMS	Amazon SNS (emergency)	$10	$20	$40	Egypt: ~$0.02/SMS

Category	Service	10K MAU	50K MAU	100K MAU	Notes
Maps	Google Maps Platform	$80	$320	$700	Geocoding + Maps SDK
Content mod	Manual review + user reporting	$0	$0	$0	No external service at v1.0 — add Rekognition post-launch if needed
DNS + TLS	Route 53 + ACM	$10	$10	$10	Fixed cost
Logs	CloudWatch Logs	$30	$80	$200	30-day retention
WAF	AWS WAF	$10	$30	$60	Managed rule groups
Background checks	Checkr	$300	$300	$300	Per-nanny, one-time cost
Support	Zendesk Growth	$149	$400	$800	Agent licensing
Monitoring	Datadog APM + Infra	$300	$800	$1,800	Full observability
Mobile CI/CD	EAS Build (Expo)	$99	$99	$99	Fixed plan
Payments	Paymob	1.75%+EGP 3/txn	1.75%+EGP 3/txn	1.5%+EGP 2/txn (negotiated vol.)	Scales with GMV; no monthly fee
TOTAL (excl. Paymob %)		**~$1,875–$2,275**	**~$4,125–$5,045**	**~$8,237–$10,037**	Range reflects compute + database option chosen

Cost Optimization: (1) Purchase 1-year Reserved Instances for RDS/Aurora and ElastiCache — saves 35–40%. (2) Aurora Serverless v2 scales to minimum ACUs overnight (Egypt timezone app) — saves ~60–70% on off-peak hours. (3) Turborepo remote cache on S3 cuts CI compute time by 60–80%. Applying these: 50K MAU target cost drops to ~$2,800–3,500/month depending on compute and database options chosen.

9. Architecture Summary

Monorepo	Mobile — Expo	AWS + Paymob
• Turborepo + pnpm workspaces • apps/mobile — React Native, Expo bare • apps/backend — modular monolith + ws-hub + video • packages/ — types, db, paymob, api-client, ui • infra/ — AWS CDK in TypeScript • One language (TypeScript) across entire repo • Claude Code reads full context in one session • Turborepo --filter runs only affected packages in CI	• React Native bare workflow (apps/mobile) • Expo Router — file-based navigation • iOS Simulator: press i in Expo CLI • Dev build: eas build --profile development • Staging: EAS Build → TestFlight • Production: EAS Build → App Store • OTA updates: eas update (no store review) • eas.json: dev / staging / production profiles	• API Gateway HTTP API — managed entry point • EKS — all backend services (apps/backend) • Aurora PostgreSQL Multi-AZ — primary DB • ElastiCache Redis • PostgreSQL full-text + PostGIS for search • S3 — media storage (required); CloudFront CDN — optional, add when media volume grows • Secrets Manager — all secrets incl. Paymob • Paymob: 3-step flow, 5 integrations, EGP • HMAC-SHA512 webhook; hybrid wallet checkout

React Native + Node.js/TypeScript + AWS API Gateway + Paymob + AWS CDK + Turborepo Monorepo. Single language (TypeScript) across mobile, backend, and infrastructure. Solo developer + agentic development ready.